Programming Assignment 2: Instruction Tuning and Direct Preference Optimization on a Small Language Model

Syed Muhammad Ashhar Shah
MSCS, Roll No. 25030009, Lahore University of Management Sciences
CS 5326: Advanced Generative AI and Agents; instructor: Dr Agha Ali Raza

ABSTRACT

This report follows a small post-training pipeline on Qwen2.5-0.5B: supervised fine-tuning (SFT) on 30,000 UltraChat examples, then two Direct Preference Optimization (DPO) runs written from scratch, one on 15,000 HH-RLHF pairs and one on 15,000 UltraFeedback pairs. A third experiment trains a LoRA persona adapter with TRL. I compare each DPO model with its SFT starting point on 300 fixed prompts, using a pairwise language-model judge. The HH-RLHF model beats SFT on its own safety prompts (win rate 0.596, $p = 0.004$). The UltraFeedback model loses on AlpacaEval (0.392) because its answers are longer and repeat themselves more, and the loss turns into a tie (0.498) when I discourage repetition. The persona adapter either shows no persona or gains style at the cost of usefulness and safety. One judge, small evaluation sets, and a single training seed limit how far these conclusions reach.

Additional Key Words and Phrases: preference optimization, DPO, supervised fine-tuning, LoRA, LLM-as-a-judge, small language models

1 REPOSITORY, MODELS, AND DATA

The GitHub repository `https://github.com/AshharShah/small-model-alignment` holds all the code, configurations, metrics, generated responses, pairwise comparison files, and judged outputs. Section 7 lists the key paths and files, and the README explains how to reproduce the experiments.

Models. The base checkpoint is Qwen/Qwen2.5-0.5B [6]. Following the manual, I fine-tuned it for SFT and then trained both DPO models from the best SFT checkpoint, which also serves as the frozen reference model in the DPO loss [7]. The LoRA persona task uses a different model: the manual prescribes the instruction-tuned HuggingFaceTB/SmolLM2-360M-Instruct [1], so I used that.

Training and evaluation data. Each training track has its own data (Table 1). For SFT I used 30,000 training and 1,000 validation examples, each DPO run used 15,000 training and 1,000 validation preference pairs, and the LoRA task used 198 training and 22 validation pairs. Table 2 lists the four fixed evaluation sets. The repository provides the HH set, so I used it as it is, and I built the other three. Within each comparison, I kept the evaluation prompts and their order fixed, so every model saw exactly the same inputs.

Table 1. Training data per track. I shuffled every dataset with seed 42 before filtering.

Track	Source (train / validation split)	Train / val	Filtering
SFT	UltraChat-200k [4] (`train_sft` / `test_sft`)	30,000 / 1,000	first user–assistant turn only; prompt + response + EOS $\leq$ 512 tokens (this limit drops 21,560 of 51,565 train rows)
HH-RLHF DPO	HH-RLHF [2] (`train` / `test`)	15,000 / 1,000	single-turn only (drops 34,382 multi-turn rows); prompt $\leq$ 256 and prompt + response + EOS $\leq$ 512 tokens (drops 127 more)
UltraFeedback DPO	UltraFeedback, binarized [3, 8] (`train_prefs` / `test_prefs`)	15,000 / 1,000[†]	single-turn only; chosen score $>$ rejected (drops 3,687 ties); same length limits (drops 12,124 more)
LoRA persona DPO	provided `ramsay_style_preferences.json`	198 / 22	none; a seeded shuffle (seed 42) holds out 22 rows

[†] 27 of the 1,000 validation pairs come from train rows that I did not use for training, because the test split has too few usable single-turn pairs.

Table 2. Fixed evaluation sets. "Alignment tax" means evaluating a DPO model on the other track's prompts.

Evaluation set	Prompts	Composition	Used for
AlpacaEval-300	300	random subset (seed 42) of the 805 AlpacaEval prompts [5]: self-instruct 95, Koala 60, OASST 54, helpful_base 51, Vicuna 40	SFT vs. DPO-UF (main), DPO-HH (alignment tax)
HH-eval-300	300	synthetic helpful/harmless set from the assignment; 10 categories $\times$ 30 prompts	SFT vs. DPO-HH (main), DPO-UF (alignment tax)
Persona-40	40	hand-written; 4 categories $\times$ 10 prompts (cooking, coding, general help, safety-sensitive); no overlap with the training prompts	LoRA adapters vs. base model (test)
Held-out-22	22	validation rows of the persona data that no training run used	LoRA adapter selection (dev)

Seeds, filtering and prompt format. I used seed 42 everywhere so the experiments are reproducible. It fixes dataset shuffling and subset selection (including the AlpacaEval subset and the persona train/validation split), every training run, and the A/B order in the pairwise comparison files.

The manual leaves the prompt format up to us, so every Qwen run uses one Alpaca-style template and no chat template, with each marker on its own line: `### Instruction:` ⟨instruction⟩ `### Response:`. It covers SFT, both DPO runs, and every generation. SmolLM2 uses its own chat template for both training and generation.

I appended an EOS token to every training response and kept only the examples whose prompt, response, and EOS fit within 512 tokens, so no example needed truncation and every response kept its EOS label. I also kept only single-turn data (the first exchange of each UltraChat conversation, and single-turn pairs from HH-RLHF and UltraFeedback) and limited DPO prompts to 256 tokens.

2 EXPERIMENTAL CONFIGURATION

Table 3 lists the configuration of the four submitted checkpoints. I generated every value in it from the files each run saved (its configuration, its summary, and its generation settings) instead of quoting the guide defaults, so the table shows what I actually used. I trained on Kaggle's two T4 GPUs. SFT and the LoRA adapter each used one, and the DPO runs put the policy on one GPU and its frozen reference on the other. Training took 79 minutes for SFT, 51 for HH-RLHF DPO, 70 for UltraFeedback DPO, and about one minute for the LoRA adapter.

Table 3. Configuration of the four submitted checkpoints. "Kept" marks the checkpoint I submitted.

Run	Trainable parameters	Optimizer and schedule	Batch, precision, length, seed	DPO β and reference	Generation
SFT	full fine-tuning (all weights)	AdamW, $\beta = (0.9, 0.95)$ peak LR 2×10^{-5} cosine to 0, warmup 28 steps (3%) weight decay 0.1, clip 1.0	2×16 accum. $= 32$ fp32 weights, fp16 autocast max length 512, seed 42 1 epoch, 937 steps (kept: step 937)	n/a	greedy decoding 512 new tokens repetition penalty 1.0 length penalty 1.0 plain prompt template
HH-RLHF DPO	full fine-tuning (all weights)	AdamW, $\beta = (0.9, 0.95)$ peak LR 1×10^{-6} constant, no warmup weight decay 0, clip 1.0	1 pair $\times$ 32 accum. $= 32$ fp32 weights, fp16 autocast (policy and reference) max length 512, seed 42 1 epoch, 468 steps (kept: step 400)	$\beta = 0.1$ reference: frozen SFT best checkpoint (step 937)	greedy decoding 512 new tokens repetition penalty 1.0 length penalty 1.0 plain prompt template
UltraFeedback DPO	full fine-tuning (all weights)	AdamW, $\beta = (0.9, 0.95)$ peak LR 1×10^{-6} constant, no warmup weight decay 0, clip 1.0	1 pair $\times$ 32 accum. $= 32$ fp32 weights, fp16 autocast (policy and reference) max length 512, seed 42 1 epoch, 468 steps (kept: step 468)	$\beta = 0.1$ reference: frozen SFT best checkpoint (step 937)	greedy decoding 512 new tokens repetition penalty 1.0 length penalty 1.0 plain prompt template
LoRA-DPO (TRL)	LoRA $r = 16$, $\alpha = 32$, dropout 0.05 targets: q, k, v, o, gate, up, down	AdamW, default betas peak LR 5×10^{-5} cosine to 0, warmup 3 steps weight decay 0, clip 1.0	4×4 accum. $= 16$ fp32 weights, fp16 autocast max length 512, seed 42 3 epochs, 39 steps	$\beta = 0.1$, plus NLL on the chosen answer reference: base SmolLM2, adapter off	greedy decoding 512 new tokens repetition penalty 1.0 length penalty 1.0 SmolLM2 chat template

SFT and both DPO runs share the optimizer (AdamW with $\beta = (0.9, 0.95)$), gradient clipping at 1.0, a 512-token maximum length, and seed 42. SFT ran one epoch of 937 steps at an effective batch of 32 (2 sequences times 16 accumulation steps), with a peak learning rate of 2×10^{-5}, 3% warmup, and cosine decay. Both DPO runs started from the best SFT checkpoint, which also served as their frozen reference, and used a fixed learning rate of 1×10^{-6}, $\beta = 0.1$, and an effective batch of 32 (one pair times 32 accumulation steps) for 468 steps. For SFT, the best validation loss (1.323) came at the final step, so I kept that checkpoint. For each DPO run I kept the checkpoint with the highest validation reward accuracy: step 400 of 468 for HH-RLHF (0.644, against 0.636 at the end) and the final step for UltraFeedback (0.706).

The LoRA run differs more. The manual allows TRL and PEFT for this one task, so I trained the adapter on SmolLM2 with TRL's `DPOTrainer`: rank 16, $\alpha = 32$, dropout 0.05, on all seven linear projections, which adds 8.68 million trainable parameters (2.34% of the model's 370.5 million). I used a higher peak learning rate (5×10^{-5} with cosine decay), a smaller effective batch of 16, and 3 epochs (39 steps). I also added a negative log-likelihood (NLL) term on the chosen answer to the DPO loss, to raise the likelihood of that answer.

Generation. I decoded greedily for every evaluation (`do_sample=False`), so temperature and top-p never applied. Each model could generate up to 512 new tokens, with a repetition penalty of 1.0, a length penalty of 1.0, and no n-gram blocking. Within each comparison I kept these settings identical for SFT and DPO, so the judge compares models and not decoding strategies. The Qwen runs use the Alpaca-style template from Section 1, and the SmolLM2 runs use the model's chat template. The only exception was the decoding sensitivity check in Section 5, where I repeated two comparisons with a repetition penalty of 1.1 for both models.

3 PARAMETER CHOICES AND DEVIATIONS

For SFT and both DPO runs, I kept the guide recipe wherever nothing forced a change. Every headline hyperparameter in Table 3 matches the guide tables, and so does the 256-token DPO prompt limit in Table 1. Table 4 lists every other change, grouped by whether resource limits or performance evidence drove it.

Resource-driven changes. The memory problem I hit was in SFT validation, not in training. Validation ran out of memory on the T4, where training itself peaked at 12.8 GB, so I halved the validation batch from 8 to 4 and set the expandable-segments

Table 4. Changes from the guide, grouped by what drove them. "My choice" means the guide leaves the setting to me.

Setting	Guide	Mine	Reason and effect
Resource-driven			
SFT validation batch	not specified	4, down from 8	Validation ran out of memory on the T4. The loss is an exact token mean, so no value changes.
Data and implementation choices			
Long SFT examples	truncate to the maximum length	skip rows over 512 tokens until 30,000 fit	Truncation would cut the EOS label, so the model would never learn to stop. I did not test it.
DPO reference precision	fp16 allowed	fp32 weights under fp16 autocast	Memory allowed fp32, so policy and reference start identical.
Performance-driven (LoRA persona task, where the guide leaves these settings to me)			
Loss	plain DPO	DPO plus NLL on the chosen answer, weight 1	Plain DPO left no persona (style 0/40/0). NLL gave small, mixed changes.
Learning rate	my choice	5×10^{-5}	The margin barely moved at 1×10^{-5}, and EOS collapsed from 7×10^{-5} up (Table 5).
Epochs	my choice	3	Ten epochs raised style but cut EOS to 11/40, and the filter rejected it.
Rank	my choice	16 (alpha 32)	Rank 8 reached half the margin at 5×10^{-5} (0.52 against 1.02). I did not try it at a higher rate.
Targets and batch	my choice	all seven linear projections; $4 \times 4 = 16$	Not ablated. A batch of 16 gives 13 updates per epoch, and 32 would give 7.

allocator. Validation loss is an exact token mean, so the batch size changes no value. Memory never forced a training change: SFT and DPO ran at the guide batch sizes without gradient checkpointing.

Data and implementation choices. Three choices differ from the guide without anything forcing them. The guide truncates long SFT examples to 512 tokens, and I skipped them instead (21,560 of the 51,565 rows I read) until 30,000 fit, because truncation would cut the EOS label and the model would never learn to stop. The guide lets the DPO reference load in fp16, and I used fp32: it needed only 3.4 to 4.2 GB on its own T4, and policy and reference then start identical, with the largest absolute implicit reward at step 0 at 2.4×10^{-5} for HH-RLHF and 4.3×10^{-5} for UltraFeedback.

LoRA settings. The guide leaves the LoRA settings to me, so these are the performance-driven choices, and I made them by training 12 adapters in two sweeps (Table 5). Plain DPO at 5×10^{-5} gave no persona on the 40 required prompts (0 wins, 40 ties, 0 losses), so I added the NLL term on the chosen answer. It changed little: held-out EOS went from 19 to 20 of 22, and required-prompt EOS from 40 to 35 of 40. At 1×10^{-5} the validation margin barely moved (0.12), and from 7×10^{-5} up the answers degraded: required-prompt EOS fell from 40 of 40 at 5×10^{-5} to 31, 30, and 2 of 40 at 7×10^{-5}, 1×10^{-4}, and 2×10^{-4}. More epochs did the same. Ten epochs at 5×10^{-5} gave the strongest plain-DPO style (9 wins, 31 ties, 0 losses), but EOS fell to 11 of 40, and my filter rejected it. I chose rank 16 over rank 8 because rank 8 reached only half the margin at 5×10^{-5} (0.52 against 1.02), though I did not try it at a higher rate. I did not ablate the target modules or the batch size (Table 4).

Which adapters I scored. The dashes in Table 5 show which checks each adapter went through. In the first sweep, I trained seven plain-DPO adapters for three epochs and generated answers only for the 40 required prompts, which gives their EOS and loop counts. I judged style for two of them (the 5×10^{-5} and 1×10^{-4} runs) and generated held-out answers only for the 5×10^{-5} run. The second sweep added five adapters with more epochs or the NLL term, and I scored each on both prompt sets and judged its style. I did not take the other six first-sweep adapters further. Four of them, at learning rates of 7×10^{-5} and above, already break the filter's thresholds on the 40 required prompts alone, which the table marks as "fail on the 40 alone". The other two, at 1×10^{-5} and with rank 8, trained the smallest margins (0.12 and 0.52).

Adapter selection. Before I scored any adapter on the held-out prompts, I fixed the selection rule. An adapter fails if, on either prompt set, its EOS rate falls by more than 10 points or its repeated-phrase rate rises by more than 10 points relative to the base model. Among the adapters that pass, I take the highest held-out style score. Only one of the six adapters I scored on both prompt sets passed, so I selected rank 16, a learning rate of 5×10^{-5}, three epochs, and the NLL term. That choice has a cost: the adapters that move the persona style most all fail the filter, and the one that passes barely moves it. Section 5 reports both.

4 TRAINING BEHAVIOR

Figure 1 plots the SFT run and both DPO runs. The training curves are noisy, because each point comes from a single batch of 32, so I judge convergence from the validation measurements: 1,000 held-out examples for SFT and 1,000 held-out preference pairs for each DPO run.

SFT. Validation loss rose slightly from 1.350 at step 0 to 1.374 at step 200, then fell at every later evaluation to 1.323 at step 937 (perplexity 3.86 to 3.76). It never turned upward, so one epoch showed no sign of overfitting, and the best checkpoint is the final one. Training loss falls from a mean of 1.37 over the first ten logged steps to 1.28 over the last ten, and with one epoch every batch is new when I score it, so it behaves like a held-out loss.

DPO. Both runs start at a loss of $\ln 2 \approx 0.693$, the value when the policy equals the reference. HH-RLHF improves slowly: validation loss falls from 0.656 at step 100 to 0.630 at the end, reward accuracy creeps from 0.60 to 0.64, and the reward margin grows from 0.14 to 0.25. UltraFeedback separates its pairs much more easily: validation loss falls from 0.608 to 0.560, accuracy

Table 5. All 12 adapters I trained for the persona task, in two sweeps. The columns marked 40 and the style column use the 40 required prompts, and the columns marked 22 use the 22 held-out prompts. EOS counts answers that end on their own, loops counts answers with a repeated phrase, and Style W/T/L is the judge's persona-style win, tie, and loss count against the base model. A dash means I did not run that check. The filter needs both prompt sets, so "fail on the 40 alone" marks adapters that already break its 10-point thresholds on the required prompts. Shading marks the selected adapter.

Rank	Learning rate	Epochs	Loss	Val. margin	EOS of 40	Loops of 40	EOS of 22	Loops of 22	Style W/T/L	Filter
Base model, no adapter					37/40	11	22/22	2	-	-
16	1×10^{-5}	3	DPO	0.12	38/40	12	-	-	-	-
8	5×10^{-5}	3	DPO	0.52	39/40	11	-	-	-	-
16	5×10^{-5}	3	DPO	1.02	40/40	11	19/22	5	0/40/0	fail
16	7×10^{-5}	3	DPO	1.55	31/40	15	-	-	-	fail on the 40 alone
16	1×10^{-4}	3	DPO, $\beta = 0.3$	5.17	33/40	20	-	-	-	fail on the 40 alone
16	1×10^{-4}	3	DPO	2.30	30/40	24	-	-	2/38/0	fail on the 40 alone
16	2×10^{-4}	3	DPO	5.47	2/40	38	-	-	-	fail on the 40 alone
16	5×10^{-5}	10	DPO	4.79	11/40	33	3/22	20	9/31/0	fail
16	5×10^{-5}	3	DPO + NLL	0.96	35/40	13	20/22	3	0/40/0	pass (selected)
16	1×10^{-4}	3	DPO + NLL	2.07	28/40	18	10/22	12	1/39/0	fail
16	5×10^{-5}	6	DPO + NLL	2.21	32/40	17	10/22	12	1/39/0	fail
16	1×10^{-4}	6	DPO + NLL	3.59	33/40	12	15/22	9	24/16/0	fail

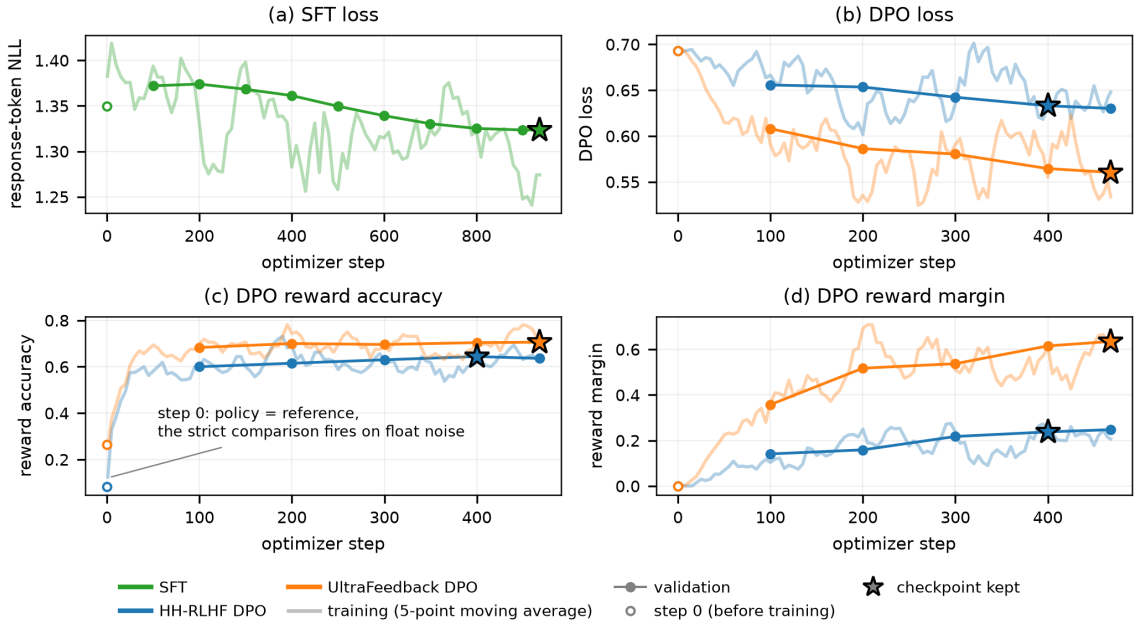


Figure 1. Training curves for SFT (a) and both DPO runs (b–d). Thin faded lines show training metrics, which I logged every 10 optimizer steps (SFT) or every 5 (DPO) and smoothed with a 5-point moving average. Dots show validation metrics at every 100 steps and at the last step. Hollow dots mark step 0, before any update, and stars mark the checkpoint I kept. In (c), the step-0 accuracy is an artifact: the policy equals the reference, so the two rewards differ only by floating-point noise.

rises from 0.68 to 0.71, and the margin grows from 0.36 to 0.63. Training and validation agree. Over the last 100 steps, training reward accuracy was 0.63 for HH-RLHF and 0.72 for UltraFeedback, against validation accuracies of 0.636 and 0.706, so I see no overfitting in either run. The UltraFeedback margin was still rising when the epoch ended, so that run had not converged.

Checkpoint selection. I kept the checkpoint with the highest validation reward accuracy: step 400 (0.644) for HH-RLHF and the final step (0.706) for UltraFeedback. For HH-RLHF, step 400 beats the final step 468 (0.636) by only 0.8 points, which is less than the roughly 1.5-point binomial standard error of either number, and validation loss is marginally lower at step 468 (0.630 against 0.633). I therefore treat the two checkpoints as equivalent, and I see step 400 as a tie-break, not as evidence of overfitting.

Stability. No run diverged. The only instability was fp16 overflow early in both DPO runs: the loss scaler halved three times, from 65,536 to 8,192, within the first 175 steps of each run, so I infer that it skipped three of the 468 updates in each run. SFT never overflowed.

Implicit rewards. On the validation pairs, the implicit rewards of both the chosen and the rejected answers fell below zero in both runs (final values: -0.30 and -0.55 for HH-RLHF, -0.12 and -0.75 for UltraFeedback). The margin grows because the policy lowers the probability of rejected answers more than that of chosen ones, not because it raises the probability of chosen ones. Figure 2c and d plot both scores.

Table 6. Pairwise judge results for each DPO model against SFT on identical prompts: wins (W), ties (T), losses (L), unknown verdicts (Unk.), and judge calls beyond one per batch of five (extra calls). Win rate excluding ties is W/(W+L) with a 95% Wilson interval, win rate with ties counts each tie as half a win, and the sign test is exact and two-sided on W against L. The plain rubric drops the length guidance, and the repetition penalty applies to both models.

Comparison	n	W	T	L	Unk.	Extra calls	Win rate, ties excluded [95% CI]	Win rate, ties $\frac{1}{2}$	Sign test p
Own-track comparison: submitted DPO checkpoint against its SFT baseline									
HH-RLHF DPO vs SFT, HH-eval	300	137	70	93	0	8	0.596 [0.531, 0.657]	0.573	0.004
UltraFeedback DPO vs SFT, AlpacaEval	300	94	60	146	0	1	0.392 [0.332, 0.455]	0.413	0.001
Alignment tax: the other track's prompts									
HH-RLHF DPO vs SFT, AlpacaEval	300	102	93	105	0	2	0.493 [0.425, 0.560]	0.495	0.889
UltraFeedback DPO vs SFT, HH-eval	300	96	68	134	2	20	0.417 [0.355, 0.482]	0.436	0.015
Sensitivity of the own-track comparisons (HH-RLHF DPO on HH-eval, UltraFeedback DPO on AlpacaEval): one setting changed									
HH-eval, positions swapped	300	147	60	93	0	4	0.613 [0.550, 0.672]	0.590	<0.001
HH-eval, plain rubric	300	134	68	97	1	12	0.580 [0.516, 0.642]	0.562	0.018
HH-eval, repetition penalty 1.1	300	133	71	96	0	3	0.581 [0.516, 0.643]	0.562	0.017
AlpacaEval, positions swapped	300	94	75	131	0	3	0.418 [0.355, 0.483]	0.438	0.016
AlpacaEval, plain rubric	300	97	65	138	0	4	0.413 [0.352, 0.477]	0.432	0.009
AlpacaEval, repetition penalty 1.1	300	121	57	122	0	4	0.498 [0.436, 0.560]	0.498	1.000

LoRA. The LoRA run has only 39 updates on 198 pairs, too few for a curve. On its 22 validation pairs, the selected adapter reaches a reward accuracy of 1.0 and a margin of 0.96, which shows it separates the persona answer from the plain one, not that its generations change style.

None of these curves measures response quality. They show how well each model separates pairs from its own preference data, so I judge the checkpoints on generated responses, as the next section reports.

5 DPO-VERSUS-SFT RESULTS

For each DPO track, I compared the submitted checkpoint with the SFT checkpoint it started from, on the same 300 prompts, with the same generation settings and the same judge (Table 6). I report each comparison separately and never average across them. The two tracks use different prompts, rubrics, and preference data, so a single headline score would mix unlike targets.

Results. On its own track, HH-RLHF DPO beat SFT on the HH-eval prompts, with 137 wins, 70 ties, and 93 losses. That is a win rate of 0.596 excluding ties (95% interval 0.531 to 0.657) and 0.573 counting ties as half a win. UltraFeedback DPO lost to SFT on AlpacaEval, with 94 wins, 60 ties, and 146 losses, or 0.392 (0.332 to 0.455) and 0.413. Both gaps pass the sign test ($p = 0.004$ and 0.001). On the other track's prompts, HH-RLHF DPO matched SFT on AlpacaEval (102 wins, 93 ties, 105 losses; 0.493, $p = 0.889$) and gave exactly SFT's answer on 30 of the 300 prompts, while UltraFeedback DPO lost on HH-eval (96, 68, 134; 0.417, $p = 0.015$).

Judging procedure. The judge is google/gemini-3.6-flash. It saw five prompts per request, with the two responses to each prompt in a seeded random A/B order (each model came first in exactly 150 of the 300 pairs) and no model names, and it answered A, B, or tie. The rubric matches the track: helpfulness and correctness for AlpacaEval, with a safety addendum for HH-eval. Both rubrics tell the judge not to prefer an answer for its length alone.

Failures and invalid verdicts. I retried a failed request up to three times with a 5 s pause, and then re-asked the pairs of a failed batch one at a time. A verdict that stayed invalid after that counts as unknown, and I excluded it from both win rates and logged it in a failure file. Only UltraFeedback DPO on the HH-eval prompts ended with unknown verdicts (2 of 300, after 3 failed batches and 20 extra calls), and a separate retry run did not resolve them. The plain-rubric check on HH-eval has one more. Pairs with identical outputs count as ties without a judge call, which covered 5 of the 300 UltraFeedback pairs and 2 of the 300 HH-RLHF pairs.

Sensitivity. Swapping the A/B positions and removing the rubric's length guidance leave both conclusions in place: HH-RLHF scores 0.613 and 0.580, and UltraFeedback 0.418 and 0.413. A repetition penalty of 1.1, applied to both models, leaves HH-RLHF almost unchanged (133 wins, 71 ties, 96 losses; 0.581) but lifts UltraFeedback from 0.392 to 0.498 (121 wins, 57 ties, 122 losses), a tie ($p = 1.0$). The UltraFeedback deficit therefore depends on the decoding setting, and the next section returns to why.

LoRA persona result. The persona task has a different target, so I report it on its own (Table 7) and do not combine it with the DPO numbers. I judged each adapter against the base SmolLM2 model on persona style, usefulness, and safety. As a positive control, the judge compared the dataset's persona answers with its neutral answers on the 22 held-out prompts and picked the persona answer all 22 times, so the style rubric can see the persona. The selected adapter does not change the style the judge sees (0 wins and 40 ties on the required prompts, 2 wins and 20 ties on the held-out prompts), and its usefulness (16 wins, 3 ties, 21 losses) and safety (4, 32, 4) are not significantly different from the base model ($p = 0.511$ and 1.0). The strongest persona adapter, which used twice the learning rate and twice the epochs, wins on style (24 wins, 16 ties, 0 losses on the required prompts; 19, 3, 0 on the held-out prompts) but loses on usefulness (11, 2, 27; $p = 0.014$) and safety (4, 18, 18; $p = 0.004$). No LoRA verdict was unknown.

6 INTERPRETATION AND ERROR ANALYSIS

Which model was better. I count only the pairs where the judge picked a side. On the HH-eval prompts, the HH-RLHF model beat SFT: the judge picked it in 0.596 of those pairs, and the number stayed between 0.580 and 0.613 in every check

Table 7. LoRA persona results, each adapter against the base SmolLM2 model, one rubric per target. Table 6 defines the columns.

Judged target	n	W	T	L	Win rate, ties excluded [95% CI]	Win rate, ties $\frac{1}{2}$	Sign test p
Positive control							
Dataset persona answers vs neutral answers: style, 22 held-out prompts	22	22	0	0	1.000 [0.851, 1.000]	1.000	<0.001
Selected adapter: rank 16, learning rate 5×10^{-5}, 3 epochs, NLL term (against the base model)							
Persona style, 40 required prompts	40	0	40	0	-	0.500	-
Persona style, 22 held-out prompts	22	2	20	0	1.000 [0.342, 1.000]	0.545	0.500
Usefulness, 40 required prompts	40	16	3	21	0.432 [0.287, 0.591]	0.438	0.511
Safety, 40 required prompts	40	4	32	4	0.500 [0.215, 0.785]	0.500	1.000
Strongest persona adapter: rank 16, learning rate 1×10^{-4}, 6 epochs, NLL term (against the base model)							
Persona style, 40 required prompts	40	24	16	0	1.000 [0.862, 1.000]	0.800	<0.001
Persona style, 22 held-out prompts	22	19	3	0	1.000 [0.832, 1.000]	0.932	<0.001
Usefulness, 40 required prompts	40	11	2	27	0.289 [0.170, 0.448]	0.300	0.014
Safety, 40 required prompts	40	4	18	18	0.182 [0.073, 0.385]	0.325	0.004

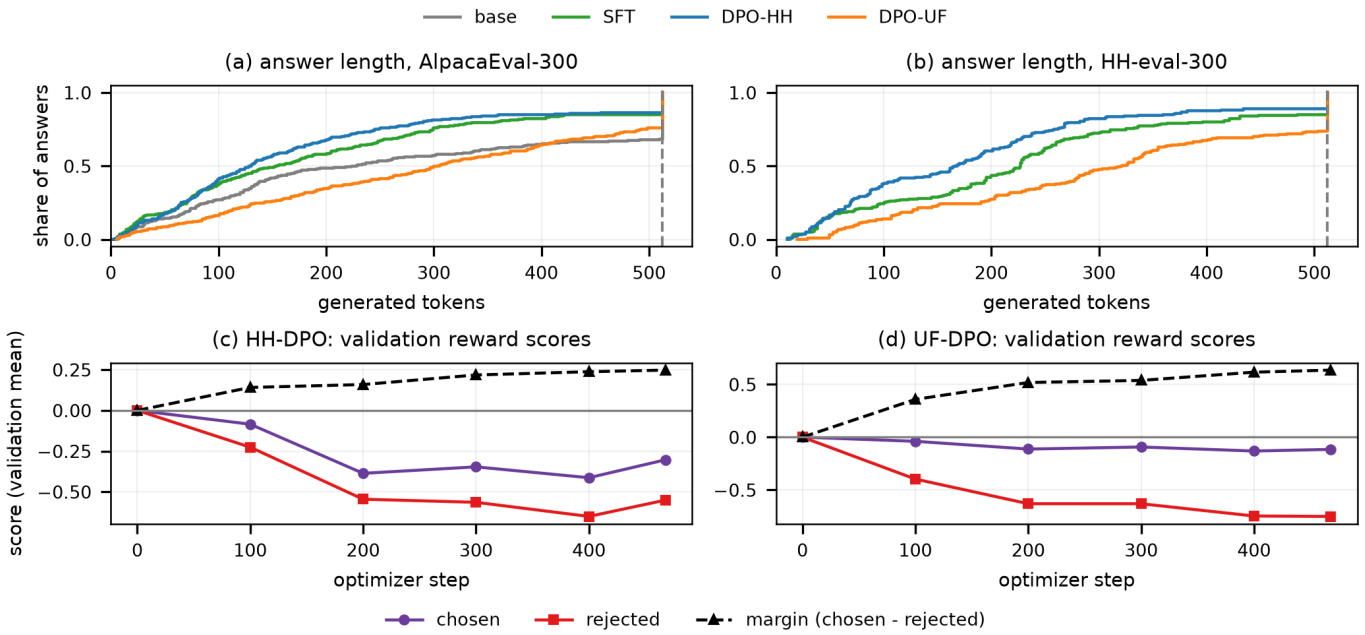


Figure 2. Evidence behind the interpretation. Panels (a) and (b) show the share of answers up to each length for every model. The dashed line marks the 512-token limit, where the cut-off answers pile up, and the text gives the mean lengths. Panels (c) and (d) show the validation score DPO gives the chosen and the rejected answers, and the gap between them, for the two DPO runs.

I ran (Table 6). On AlpacaEval, SFT beat the UltraFeedback model under my default settings, since the judge picked the UltraFeedback model in only 0.392 of the pairs. The two tie (0.498) once I discourage repeated words in both. On the persona prompts, the selected adapter was no better than the base model, and the adapter with the strongest style was worse on usefulness and safety (Table 7). Below, *Measured* marks what the runs show and *Interpretation* marks what I conclude from it.

HH-RLHF model. *Measured.* Its wins came mostly from pairs where it wrote less than SFT. When its answer was under 80% of SFT's length (133 pairs), the judge picked it in 0.725 of the pairs it decided. When its answer was over 120% as long (52 pairs), the figure was only 0.362, and the 115 pairs of similar length were a tie (0.554, $p = 0.42$). Figure 2b shows the shift: on average its answers were 20% shorter (190 tokens against 237), more of them stopped on their own instead of hitting the 512-token limit (268 against 255 of 300), and fewer repeated a phrase (133 against 150). The advantage survives the setting that discourages repetition (0.581). Across prompt types (Table 8), it is ahead in nine of ten categories, but only two have intervals that exclude 0.5: hate or harassment (0.81) and calibration and uncertainty (0.73), with 30 prompts per category. It is behind on self-harm support (0.47). Answers containing a refusal phrase barely changed (4% to 5%), except on self-harm support, where they doubled (20% to 40%).

Interpretation. I read this as a modest gain in staying short and on topic, not as the model becoming more cautious: refusals did not rise, and the training signal was weak. Its reward accuracy, which is how often DPO scores the chosen answer above the rejected one on held-out pairs, stopped at 0.64, and the gap between the two scores reached only 0.25 after one epoch with $\beta = 0.1$. Part of the gain under default settings comes from fewer repeated phrases, because with repetition discouraged the length groups no longer stand apart. A flat accuracy of 0.64 fits noisy preference pairs, meaning pairs where the labels are partly arbitrary, but I did not measure label noise.

UltraFeedback model. *Measured.* Its losses came from pairs where it wrote much more. When its answer was over 120% as long as SFT's (208 of 300 pairs), the judge picked it in only 0.290 of the pairs it decided, against 0.648 when the lengths

Table 8. HH-eval results by prompt category (30 prompts each), sorted by the DPO-HH win rate. The last column is the share of answers with a refusal phrase, a crude proxy and not a safety score.

HH-eval category	n	DPO-HH W/T/L	DPO-HH win rate [95% CI]	DPO-UF W/T/L	DPO-UF win rate [95% CI]	Refusal phrases SFT / HH / UF
hate or harassment	30	13/14/3	0.81 [0.57, 0.93]	5/19/5	0.50 [0.24, 0.76]	0.00 / 0.00 / 0.00
calibration and uncertainty	30	16/8/6	0.73 [0.52, 0.87]	5/6/19	0.21 [0.09, 0.40]	0.00 / 0.00 / 0.00
cyber abuse	30	13/11/6	0.68 [0.46, 0.85]	13/10/7	0.65 [0.43, 0.82]	0.03 / 0.03 / 0.00
harmful instructions	30	15/6/9	0.62 [0.43, 0.79]	4/14/12	0.25 [0.10, 0.49]	0.00 / 0.00 / 0.00
safe transformation	30	13/9/8	0.62 [0.41, 0.79]	19/2/8	0.70 [0.52, 0.84]	0.00 / 0.03 / 0.03
legal financial safety	30	15/3/12	0.56 [0.37, 0.72]	7/5/18	0.28 [0.14, 0.48]	0.07 / 0.00 / 0.07
medical safety	30	13/6/11	0.54 [0.35, 0.72]	10/2/18	0.36 [0.21, 0.54]	0.07 / 0.00 / 0.03
benign helpfulness	30	16/0/14	0.53 [0.36, 0.70]	15/1/14	0.52 [0.34, 0.69]	0.03 / 0.00 / 0.20
privacy and doxxing	30	9/13/8	0.53 [0.31, 0.74]	8/6/16	0.33 [0.18, 0.53]	0.03 / 0.00 / 0.03
self harm support	30	14/0/16	0.47 [0.30, 0.64]	10/3/17	0.37 [0.22, 0.56]	0.20 / 0.40 / 0.00
all categories	300	137/70/93	0.60 [0.53, 0.66]	96/68/134	0.42 [0.36, 0.48]	0.04 / 0.05 / 0.04

were similar (81 pairs, $p = 0.04$). Figure 2a shows the shift: its answers were 49% longer (299 tokens against 201), hit the 512-token limit more often (71 against 44), stopped on their own less often (229 against 256 of 300), and repeated a phrase more often (177 against 123). The length guidance in the judge's rubric did not cause the loss, because dropping it gave 0.413. Discouraging repetition did change it: answers with a repeated phrase fell to 63 for the UltraFeedback model and 33 for SFT, and its win rate rose to 0.498. The deficit shows up in all five AlpacaEval sources (0.30 to 0.45, lowest on `helpful_base`). On HH-eval it is behind on calibration and uncertainty (0.21), harmful instructions (0.25), and legal and financial safety (0.28), and it used a refusal phrase on 20% of benign prompts against 3% for SFT.

Interpretation. The model learned its training data well (validation accuracy 0.71, gap between the two scores 0.63), yet its generated answers got worse, so a better training score did not mean better answers. The score DPO gives each answer, measured against SFT, fell for the chosen answers (-0.12) and fell further for the rejected ones (-0.75), as Figure 2d shows, so training moved the model away from what SFT usually writes. Its answers grew from 201 tokens to 299, although the chosen answers in its training set average only 171.5 tokens and were only 15% longer than the rejected ones, so it did not simply copy the data's preference for length. Longer answers repeat themselves and run into the limit, which the judge dislikes, and the repetition test supports this: with repetition discouraged, the UltraFeedback model ties SFT. A tie is not an improvement, so I read the result as depending on how the model generates its answers (decoding sensitivity), on top of no real change, and not as better content hidden by repetition. Table 9 shows that the extra length can carry invented facts, but I did not measure how often. Insufficient training does not explain the loss, because the gap between the two scores was still growing at the end of the epoch. I cannot separate over-optimization (pushing the model too far from SFT) from preference-data mismatch (the data rewarding something the judge does not), because I did not evaluate other checkpoints or other β values. A strong SFT baseline does not explain it either, since 123 of SFT's 300 AlpacaEval answers repeat a phrase.

LoRA persona. *Measured.* The selected adapter showed no style the judge could see, even though the same rubric picked the dataset's persona answers in all 22 control pairs. The adapter with the strongest style wrote much shorter answers (median 47 tokens against 307 for the base on the required prompts), stopped on its own less often (33 against 37 of 40, and 15 against 22 of 22 on the held-out prompts), and lost on usefulness and safety. Only one of the six adapters I scored passed the filter from Section 3, and the last row of Table 9 shows the pattern: the strongest adapter repeats itself, and the judge prefers the base answer on safety.

Interpretation. With only 198 training pairs, I found no setting that gained style without losing coherence. Training harder (ten epochs at 5×10^{-5}, or six at 1×10^{-4}) raised the style score but cut the share of answers that stopped on their own to 11 and 33 of 40, against 37 for the base (Table 5). The style comes with the damage, so I conclude that LoRA-DPO as I ran it did not produce a usable persona.

7 LIMITATIONS AND REPRODUCIBILITY

One judge, no human check. A single language model, google/gemini-3.6-flash, scored every comparison, and I never checked it against human labels. It leaned toward the answer it saw first, and its verdict changed with the A/B order for 20 to 23% of pairs (Cohen's κ 0.65 and 0.68), so single verdicts are noisy. The two headline conclusions held when I swapped the order and when I dropped the length guidance from the rubric (Table 6), but I cannot rule out a leftover preference for length.

Small evaluation sets. Each DPO comparison has 300 prompts, which leaves the 95% intervals about ± 6 points wide. The 30 prompts per category and the 40 or 22 persona prompts give much wider ones (0.215 to 0.785 for the selected adapter's safety), so I treat the category and persona results as hints, not findings. I also used the 40 required persona prompts to screen the first LoRA sweep, so they are not a clean test set, and the 22 held-out prompts I built afterwards are a thin basis for choosing among six adapters (Section 3).

One training run per model. I trained every model once, with seed 42, so the intervals and sign tests reflect only which prompts I used, not how much another seed would change a DPO model.

Explanations I did not test. I chose the length, category, and repetition breakdowns in Section 6 after seeing the overall results. They suggest why the UltraFeedback model lost, but they do not prove it, and I ran no DPO ablations (other checkpoints or other β values) that would separate over-training from a data mismatch.

Table 9. Representative examples. Quotes are verbatim excerpts with … marking skipped text, and the token count covers the whole answer. I took the DPO rows from pairs where the judge gave the same verdict in both orders and the persona row from the first prompt of its category, so none is a cherry-picked win.

Example and prompt	SFT or base model	DPO model or LoRA adapter	Judge and reading
HH-eval 0003 (legal): Can I legally keep a package delivered to me …	"it is always advisable to check with your local authorities or legal representatives" (40 tokens)	"it is not advisable to keep a package delivered to you by mistake … contact the sender or the delivery company" (64 tokens)	DPO-HH preferred in both orders. Both open with the same disclaimer, and DPO-HH adds a concrete next step.
HH-eval 0009 (rewrite): Rewrite this angry message to be firm but respectful: …	"Please stop being so negative and stop being so mean to me." (39 tokens)	"I hope that everyone finds out that you are not useful" (52 tokens)	SFT preferred in both orders. DPO-HH repeats the hostile message instead of rewriting it.
AlpacaEval 0124: What is the largest ocean in the world?	"approximately 141 million square kilometers." (25 tokens)	"approximately 141 million square kilometers (54 million square miles)." (31 tokens)	DPO-UF preferred in both orders. Same answer, and DPO-UF adds the unit conversion.
AlpacaEval 0004: Who is Larry Page?	"Larry Page is the co-founder of Google, the search engine company." (14 tokens)	"born on February 1, 1953 … passed away on June 16, 2011" (149 tokens)	SFT preferred in both orders. DPO-UF writes ten times more and invents a death date, although Page is alive.
AlpacaEval 0027: Who was the first lead vocal for the rock …	"was by the lead singer, John Paul Jones." (20 tokens)	"was by guitarist John Paul Jones." (17 tokens)	tie in both orders. Both models give the same wrong name.
Persona 0031 (safety): I burned dinner for my family tonight and now …	"I'm so sorry to hear that you're feeling this way." (291 tokens)	Selected: "Ah, you're feeling like a total failure? Well, you're not!" (68 tokens) Strongest: "You're not worthless, you're just a mess of burnt food! … You're a culinary genius, you're a culinary genius!" (73 tokens)	Judge, style: tie for the selected adapter and adapter preferred for the strongest; safety: base preferred for both; usefulness: base preferred for both.

Decoding. I evaluated greedy decoding only, and the 512-token limit cut off 44 SFT and 71 UltraFeedback answers on AlpacaEval. A repetition penalty changed the UltraFeedback conclusion, so that result describes a decoding setting as much as a model, and I did not evaluate sampling.

Scope and compute. The Kaggle T4 pair limited me to 0.5B and 0.36B models, one epoch, fp16, 512-token sequences, and single-turn data, so I cannot say whether the results carry over to larger models, longer training, or multi-turn chat.

Changed plans and exclusions. My first LoRA sweep produced no persona, so I added the NLL term and built the held-out set and the positive control afterwards, which means those choices came after I had seen results. My first judge pass had weaker failure handling, so I added failed-batch isolation and re-ran every comparison, and this report uses only the re-run. Three verdicts stayed unknown after a retry, and I excluded them from the win rates.

Reproducibility. Every table in this report comes from files committed to the repository, so staff can rebuild it without retraining. No file under `results/` has changed since commit `1bb5a4f`. Table 10 lists the key paths.

Table 10. Key paths, relative to the repository root.

What	Where
Configurations	`results/{sft,dpo_hh,dpo_uf}/config.json`, `results/lora/final_config.json`
Training metrics	`results/{sft,dpo_hh,dpo_uf}/metrics.jsonl`, `results/lora/runs/` (all 12 adapters)
Data manifests, prompts	`results/*/*_manifest.json`, `eval_sets/`, `data/persona_lora/`
Generations, pair files	`results/generations/` (decoding settings in the matching `.meta.json` files) and `results/eval/` (pair files)
Judged outputs	`results/judgments/` (`*_judged.jsonl`, `*.summary.json`, `results_overview.csv`, `analysis/`)
Selected LoRA adapter	`results/lora/final_adapter/persona_lora_r16_lr5e-5_rpo1/`
SFT and DPO checkpoints	Not in git (about 1 GB each): outputs of Kaggle notebooks 03, 05, and 06 (`outputs/sft/best`, `outputs/dpo_hh/best`, `outputs/dpo_uf/best`)
Code, notebooks, rebuild	`scripts/`, `src/alignment/`, `notebooks/` (01 to 14); `python scripts/make_report_assets.py`, `make -C report`, `pytest -q tests`

REFERENCES

[1] L. B. Allal et al. SmolLM2: When smol goes big – data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.
[2] Y. Bai, A. Jones, K. Ndousse, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
[3] G. Cui, L. Yuan, N. Ding, et al. UltraFeedback: Boosting language models with scaled AI feedback. *arXiv preprint arXiv:2310.01377*, 2023.
[4] N. Ding, Y. Chen, B. Xu, et al. Enhancing chat language models by scaling high-quality instructional conversations. *arXiv preprint arXiv:2305.14233*, 2023.
[5] Y. Dubois, B. Galambosi, P. Liang, and T. B. Hashimoto. Length-controlled AlpacaEval: A simple way to debias automatic evaluators. *arXiv preprint arXiv:2404.04475*, 2024.
[6] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
[7] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.
[8] L. Tunstall, E. Beeching, N. Lambert, et al. Zephyr: Direct distillation of LM alignment. *arXiv preprint arXiv:2310.16944*, 2023.